Heart Disease Risk Classifier

End-to-end ML model development, CI/CD and production deployment

MLOps Experiential Learning Assignment · Introduction to MLOps (AIMLCZG523)

Item	Details
Code repository	<add your GitHub URL after pushing — see section 11>
Dataset	UCI Heart Disease, Cleveland subset (303 patients, 13 attributes)
Champion model	Logistic regression (C = 0.3, L2, class_weight = balanced)
Headline result	CV ROC-AUC 0.907 ± 0.019 · test ROC-AUC 0.961 · test recall 0.929
Serving	FastAPI `/predict` in Docker, Kubernetes Deployment (2 replicas) behind a LoadBalancer Service
Access	Local: `http://localhost:8000` (Docker) or the LoadBalancer IP (Kubernetes) — section 11
Video	`video/pipeline_walkthrough.mp4` in the repository

1. Problem and approach

The goal is to predict whether a patient has heart disease from routine clinical measurements, and to run that model as a reproducible, tested, containerised and monitored service. The target is binary: `num` = 0 means no disease and `num` = 1–4 (angiographic narrowing > 50 %) is treated as disease.

The work follows the MLOps lifecycle covered in the course: versioned data and code, experiment tracking, a single preprocessing-plus-model pipeline for training and serving, automated tests and quality gates in CI, an immutable container image, a declarative Kubernetes deployment with probes and rolling updates, and request-level logging and metrics. Figure 1 shows how the pieces fit together.

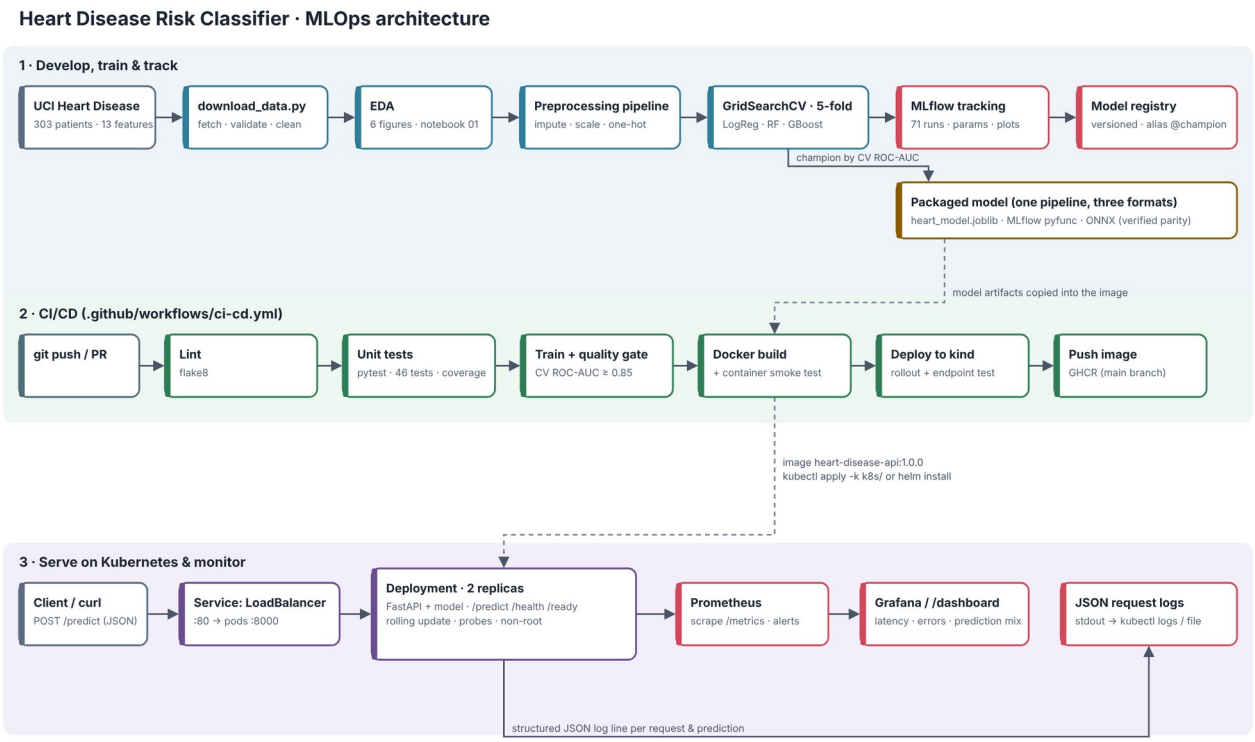


Figure 1. Solution architecture: develop and track (lane 1), CI/CD (lane 2), serve and monitor (lane 3).

2. Setup and installation

Everything runs from a clean environment with `requirements.txt` (Python 3.11 in CI, 3.12 also tested). Versions of every direct dependency are pinned, plus SQLAlchemy, whose 2.1 release breaks the MLflow 2.22 SQLite backend. The serving image uses a smaller `requirements-serve.txt`.

```
git clone <repo-url> heart-disease-mlops && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
python scripts/download_data.py        # raw + cleaned data
python -m heart.eda                    # figures
python -m heart.train                  # tuning, MLflow, packaged model
python -m pytest -v && flake8          # tests and lint
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

`make help` exposes the same steps, and `scripts/run_ci_locally.sh` runs the full CI sequence on a workstation. The setup was verified by cloning the repository into an empty directory, creating a new virtual environment with plain `pip`, deleting the data and model folders, and running every step above plus the three notebooks.

3. Data acquisition and EDA

3.1 Acquisition and cleaning

`scripts/download_data.py` downloads `processed.cleveland.data` from the UCI archive. If UCI is unreachable it falls back to the direct file URL and then to a public mirror of the same 303 records, converting its text labels back to UCI codes. It writes the URL used and the file's SHA-256 to `data/raw/SOURCE.txt`, so every run is traceable to its exact input.

Cleaning (`heart.data.clean`) coerces every column to numeric, drops duplicates and unlabelled rows, turns physiologically impossible values and unknown category codes into missing values, and binarises the target. The data has only six missing cells (`ca`: 4, `thal`: 2). They are deliberately **not** imputed in the cleaned file. Imputation

happens inside the model pipeline and is fitted on training folds only, so statistics from the test set never leak into training.

3.2 Findings

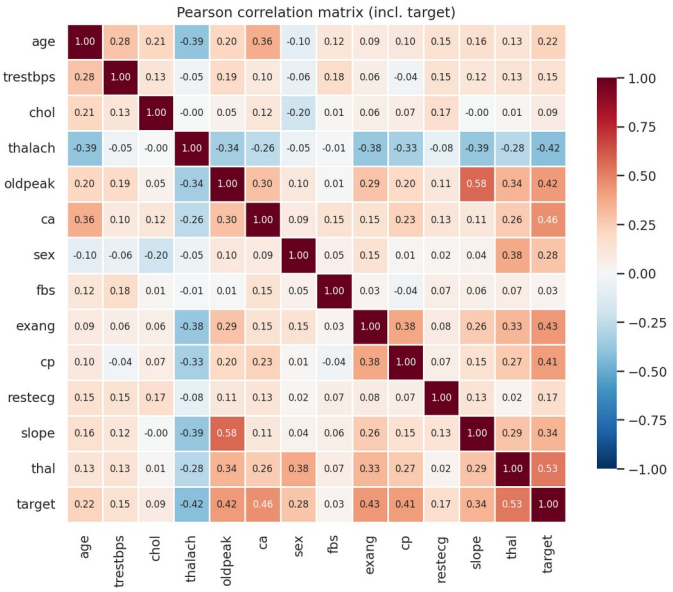


Figure 2. Correlation matrix including the target.

- **Balanced classes** (164 vs 139, 54 % / 46 %), so no resampling is needed. ROC-AUC is the selection metric because it does not depend on a threshold, and recall is reported because a missed case is the costliest error.
- **Strongest signals:** `thal` (0.53), `ca` (0.46), `exang`, `oldpeak`, `cp` (≈ 0.41–0.43) and a negative effect of maximum heart rate `thalach` (−0.42). These agree with clinical knowledge.
- `chol` (0.09) and `fbs` (0.03) carry little signal on their own. They are kept and left to regularisation.
- `cp`, `restecg`, `slope` and `thal` are **nominal codes** (thal = 3/6/7), so they are one-hot encoded rather than used as numbers.

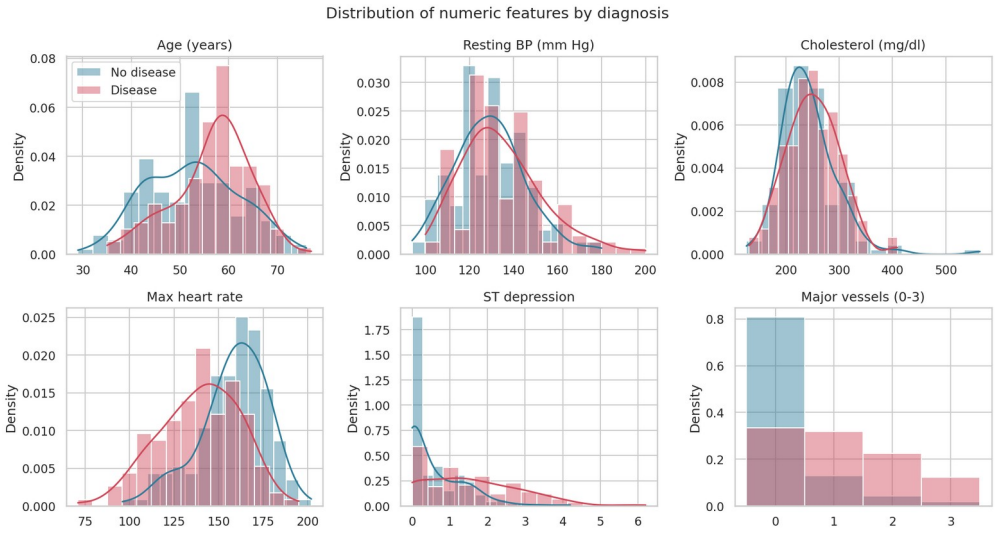


Figure 3. Numeric features by diagnosis. Disease shifts age and ST depression up and maximum heart rate down.

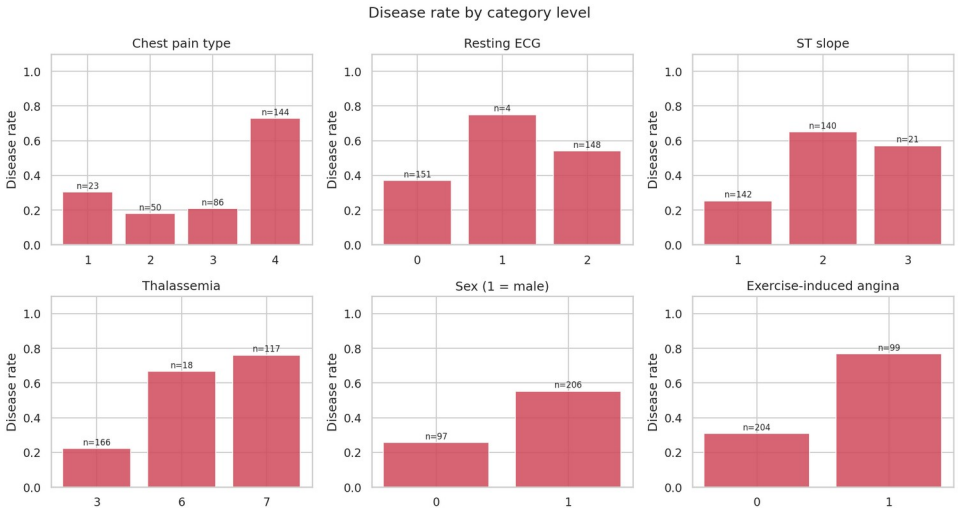


Figure 4. Disease rate per category level (n = patients in the level).

The remaining figures (class balance, missing values, outlier box plots) are in `reports/figures/` and `notebooks/01_eda.ipynb`.

4. Feature engineering and model development

4.1 Preprocessing pipeline

A single scikit-learn `ColumnTransformer` (`heart.features`) turns 13 raw inputs into 22 model features:

- 6 numeric features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca`): median imputation, then standard scaling. Scaling is switched off for tree models, which do not need it.
- 3 binary flags (`sex`, `fbs`, `exang`): most-frequent imputation.
- 4 nominal features: most-frequent imputation, then one-hot encoding with the **fixed UCI category levels** and `handle_unknown="ignore"`, so the encoding is identical in every fold and at serving time.

The transformer is the first step of the saved `Pipeline`. The API therefore applies exactly the medians, scaling statistics and categories learned in training, which removes training-serving skew by construction.

4.2 Models, tuning and selection

Data is split 80/20, stratified and seeded (242 training, 61 test). Three model families are tuned with `GridSearchCV` and 5-fold stratified CV on the training split. Each candidate is scored on accuracy, precision, recall, F1 and ROC-AUC, and the best is refitted on ROC-AUC. Validation curves in `notebooks/02_training.ipynb` set the grid ranges: logistic regression is flat for C ≥ 0.1, and random-forest training AUC reaches 1.0 for deep trees while CV AUC plateaus at depth 4–8.

Model	Grid (candidates)	Best parameters
Logistic regression	C ∈ {0.01…10} × L1/L2 × class weight (24)	C = 0.3, L2, balanced
Random forest	trees × depth × min leaf × max features (36)	200 trees, depth 4, leaf 5, sqrt
Gradient boosting	trees × learning rate × depth (8)	100 trees, lr 0.03, depth 2

The champion is chosen on **mean CV ROC-AUC**, never on the test set, which is used only once for the final estimate.

Model	CV ROC-AUC	CV accuracy	CV precision	CV recall	Test ROC-AUC	Test recall
Logistic regression	0.907 ± 0.019	0.855 ± 0.027	0.882 ± 0.070	0.801 ± 0.075	0.961	0.929
Random forest	0.901 ± 0.024	0.826 ± 0.029	0.841 ± 0.049	0.773 ± 0.087	0.956	0.929
Gradient boosting	0.879 ± 0.029	0.822 ± 0.029	0.831 ± 0.068	0.783 ± 0.085	0.954	0.929

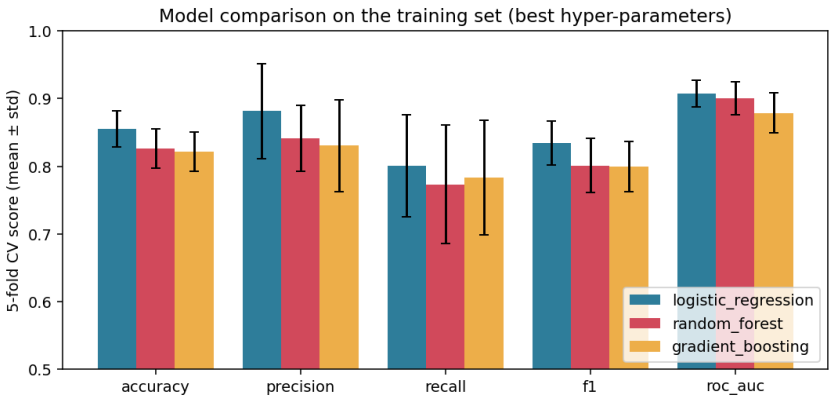


Figure 5. 5-fold CV scores of the tuned models (error bars = standard deviation across folds).

Why logistic regression wins. With 242 training rows and mostly additive, monotonic risk factors, a regularised linear model matches or beats the ensembles on CV ROC-AUC (0.907 vs 0.901 and 0.879) and has the lowest variance across folds. It is also interpretable and tiny to serve. Its test scores are higher than its CV scores, but with only 61 test patients that gap is within sampling noise, so the CV estimate is treated as the reliable one. At the 0.5 threshold it misses 2 of 28 disease cases in the test set (recall 0.93).

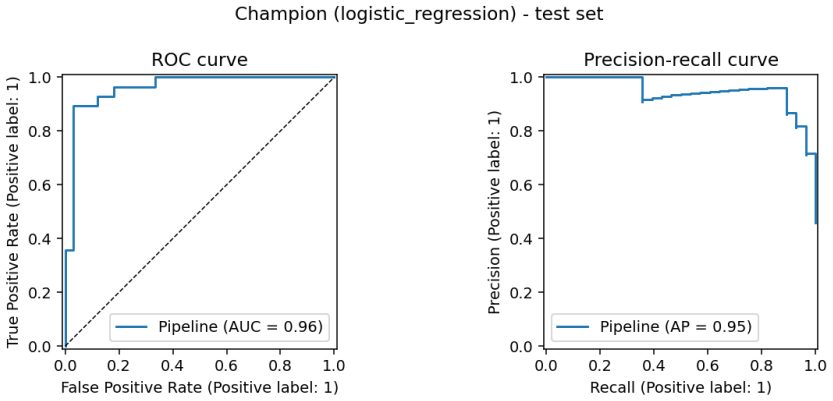


Figure 6. Champion on the held-out test set: ROC and precision-recall curves.

5. Experiment tracking (MLflow)

`heart.train` logs to an MLflow tracking store (SQLite backend, local artifact store). Every training run creates:

- **One parent run per model family:** best hyper-parameters, CV mean/std for all five metrics, test metrics, fit time, a train-vs-CV gap, the fitted pipeline with an input signature and example, and evaluation artifacts (confusion matrix, ROC/PR curves, feature importance, full `cv_results.csv`).
- **One nested run per grid candidate** (68 in total) with its parameters and CV metrics, so every configuration that was tried is tracked, not only the winners.
- **Lineage tags** on every run: git commit, SHA-256 of the cleaned data, dataset name and mode.
- **Model Registry:** the champion is registered as `heart-disease-classifier` with the alias `@champion` and a `cv_roc_auc` tag.

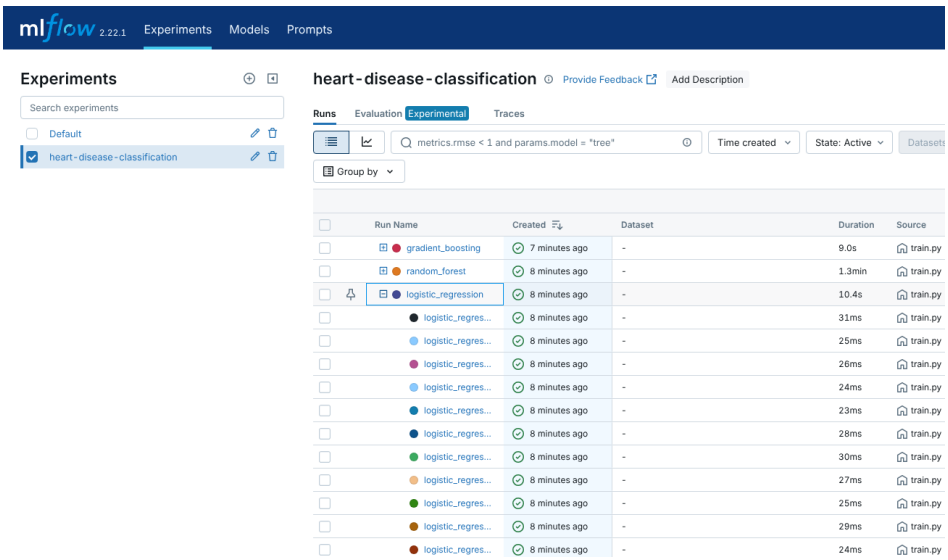


Figure 7. MLflow experiment: three parent runs, with the logistic-regression run expanded to show its nested grid candidates.

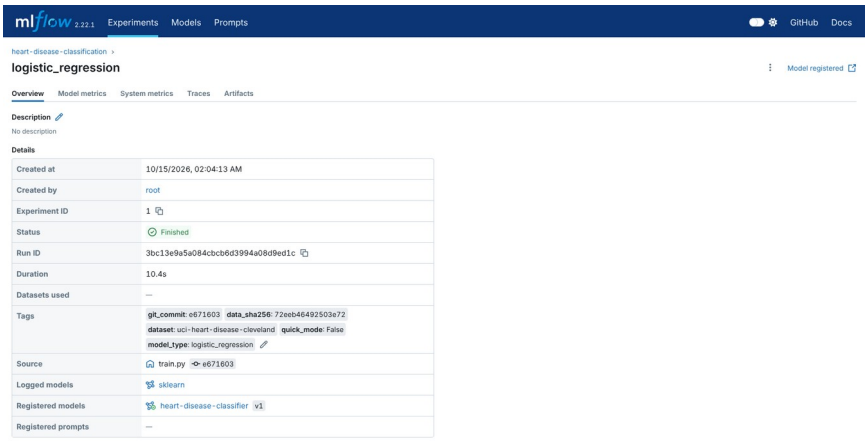


Figure 8. Champion run: lineage tags (git commit, data hash), parameters and metrics.

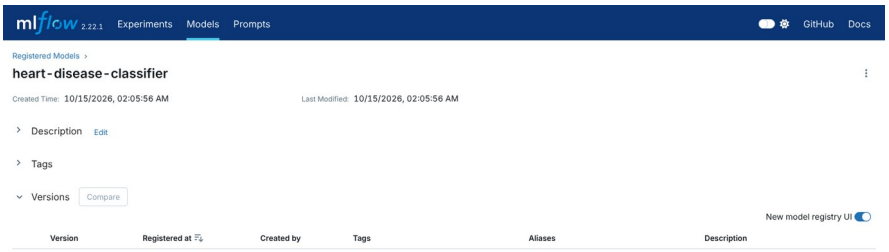


Figure 9. Model Registry: version 1 with the @champion alias.

`scripts/export_mlflow_runs.py` exports all 71 runs to `reports/mlflow_runs.csv` for reporting. More views (chart comparison, artifacts) are in `screenshots/`.

6. Model packaging and reproducibility

The full pipeline (preprocessing plus classifier) is saved in three formats:

- `models/heart_model.joblib` – the scikit-learn pipeline, loaded by the API.
- `models/mlflow_model/` – MLflow model (pyfunc + sklearn flavours) with signature, input example, `conda.yaml` and `requirements.txt`.
- `models/heart_model.onnx` – ONNX graph including preprocessing. The export is checked automatically: predictions must match scikit-learn within 1e-3, and the maximum difference observed was 9e-8.

`models/model_metadata.json` records the algorithm, parameters, CV and test metrics, decision threshold, feature list, training time, MLflow run id, git commit, data hash and library version. The API serves it at `GET`

`/model`, so any prediction can be traced back to the code, data and run that produced the model. Reproducibility also rests on: fixed random seeds for the split, CV folds and models; pinned dependency versions; the committed cleaned dataset and train/test split; and a container image that freezes the runtime.

7. CI/CD pipeline and automated testing

7.1 Tests

Test module	Tests	What is checked
test_data.py	12	UCI parsing ('?' → NaN), mirror conversion, target binarisation, duplicates, impossible values, missing columns, download fallback and failure, committed dataset shape and class counts
test_features.py	7	output shape and names, no NaN after imputation, standardisation, one-hot integrity, unseen category handling, determinism
test_model.py	10	CV ROC-AUC > 0.85, valid probabilities, metric maths, confidence for both classes, risk ordering, packaged model reload, end-to-end training run in a temp dir
test_api.py	17	health, readiness, /predict output and confidence, null ca/thal, 7 invalid-input cases → 422, batch, metrics, /stats, dashboard, 503 without a model

Coverage is 86 % overall (100 % on features and schemas, 91 % on the API). The tests earned their place during development. An injected bug that computed the confidence from the wrong class initially passed, because the test patients were all low-risk. The tests were then extended to require both classes, and the bug is now caught (Figure 12).

7.2 GitHub Actions workflow

`.github/workflows/ci-cd.yml` (validated with `actionlint`) runs on every push and pull request. Jobs depend on each other in sequence, and every step runs under `bash -euo pipefail`.

Job	Steps	Artifacts / logs
lint	flake8, with errors annotated on the PR	lint-report
test	install from requirements, pytest + coverage, summary in the run page	JUnit XML, coverage XML/HTML
train	download + clean, EDA, GridSearchCV + MLflow, **quality gate** (fail if CV ROC-AUC < 0.85), comparison table in the summary	models/, reports/, training.log, MLflow store
docker	build image, run container, `smoke_test.sh` (health, ready, predict, batch, 422, metrics), push to GHCR on main	image tarball, build and container logs
deploy-k8s	kind cluster, `helm lint` + template, apply manifests, wait for rollout, smoke test through the Service, describe/logs on failure	kubectl status, rendered Helm chart

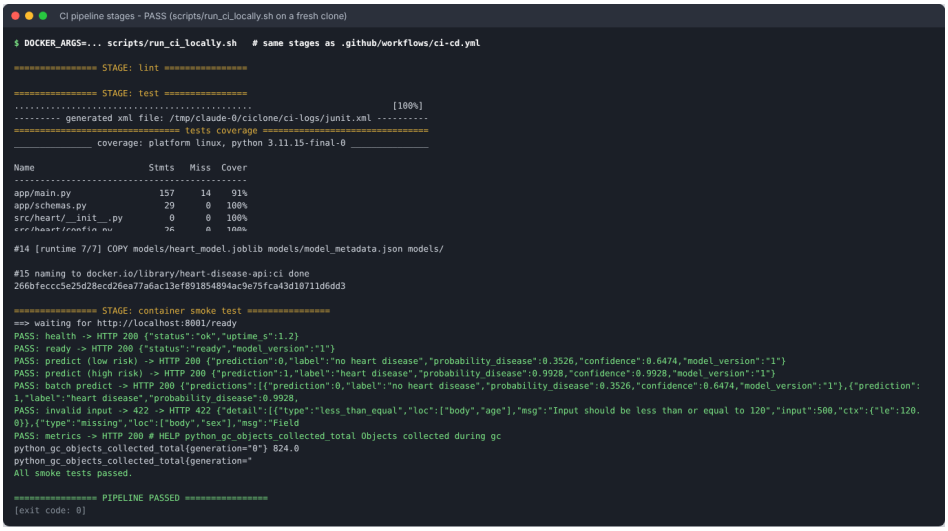


Figure 10. All pipeline stages passing on a fresh clone (scripts/run_ci_locally.sh runs the same stages as the workflow). Middle section trimmed.



Figure 11. The pipeline stops at the test stage on an injected bug, with the failing assertions and a clear stage name. Middle section trimmed.

A lint error stops the pipeline at the first stage in the same way (`screenshots/term_15_ci_fails_on_lint_error.png`).

8. Model containerisation

The FastAPI service (`app/main.py`) exposes `POST /predict`, which takes one patient as JSON and returns the prediction, label, probability of disease, **confidence** (probability of the predicted class) and model version. It also exposes `/predict/batch`, `/health`, `/ready`, `/model`, `/metrics`, `/stats` and `/dashboard`. Pydantic validates every field's range and code set, rejects unknown fields, and accepts null for `ca`/`thal`.

The Dockerfile is multi-stage. A builder stage creates a virtual environment with serving dependencies only. The runtime stage copies that environment, the package, the API and the model, runs as a non-root user (uid 10001), and defines a `HEALTHCHECK` on `/health`. The base image is a build argument (default `python:3.11-slim`).

```
docker build -t heart-disease-api:1.0.0 .
docker run -d -p 8000:8000 --name heart-api heart-disease-api:1.0.0
curl -X POST localhost:8000/predict -H 'content-type: application/json' -d
@samples/patient_high_risk.json
{"prediction":1,"label":"heart
disease","probability_disease":0.9928,"confidence":0.9928,"model_version":"1"}
```

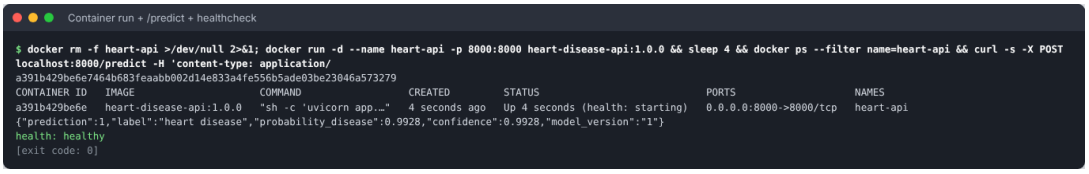


Figure 12. Container running locally: prediction with confidence, healthcheck reported healthy.

9. Production deployment (Kubernetes)

`k8s/` holds plain manifests applied with `kubectl apply -k k8s/`: a namespace, a ConfigMap, a Deployment, a LoadBalancer Service, an Ingress and an HPA. `helm/heart-api/` packages the same deployment as a Helm chart. The Deployment has the following settings:

- 2 replicas and a rolling update with `maxUnavailable: 0`.
- A **readiness probe** on `/ready`, so traffic is routed only once the model is loaded, and a liveness probe on `/health`.
- CPU and memory requests and limits.
- A read-only root filesystem, dropped capabilities and a non-root user.
- Prometheus scrape annotations.

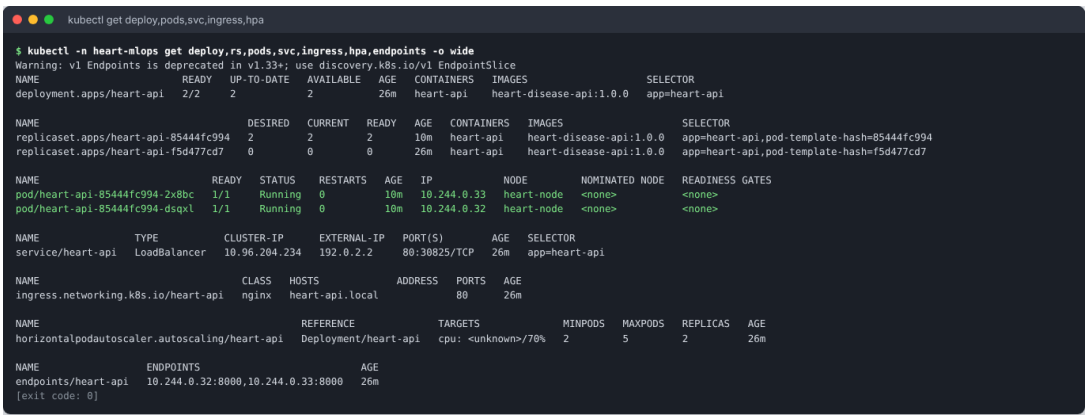


Figure 13. Deployment 2/2 ready, both pods running, LoadBalancer Service with an external IP, Ingress and HPA.



Figure 14. Endpoint verification through the LoadBalancer IP: all smoke checks pass.

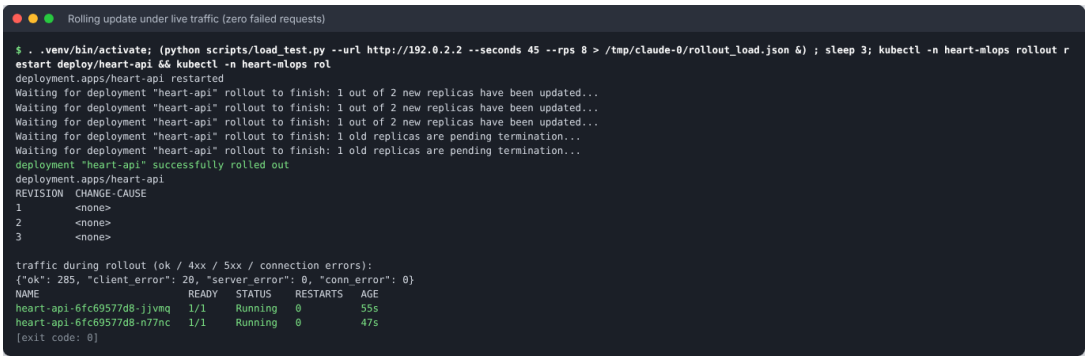


Figure 15. Rolling update under live traffic: 285 successful requests and 0 server or connection errors. The 20 4xx are deliberately malformed requests from the load generator.

A live `/predict` call through the LoadBalancer from the Swagger UI is in `screenshots/api_03_swagger_predict_executed.png`. The CI `deploy-k8s` job repeats the deployment on a fresh kind cluster for every commit.

10. Monitoring and logging

- **Logging:** middleware writes one JSON line per request (request id, method, path, status, latency, client). Each prediction is logged with its inputs, output and model version. The request id is returned in the `x-request-id` header for correlation. Logs go to stdout (read with `kubectl logs`) and optionally to a rotating file.
- **Metrics** (`/metrics`, Prometheus format): request count by endpoint and status, a latency histogram, predictions by class, a predicted-probability histogram, error count, `model_loaded` and `model_info{version}`.
- **Alerts** (`monitoring/alerts.yml`, checked with `promtool`): API down, error rate > 5 %, p95 latency > 500 ms, model not loaded, and a prediction-mix drift signal (share of positive predictions far from the 46 % training prevalence).
- **Dashboards:** a provisioned Grafana dashboard (`docker compose up`) and a built-in `/dashboard` page in the API showing traffic, latency percentiles, error rate, prediction mix and the recent request log.

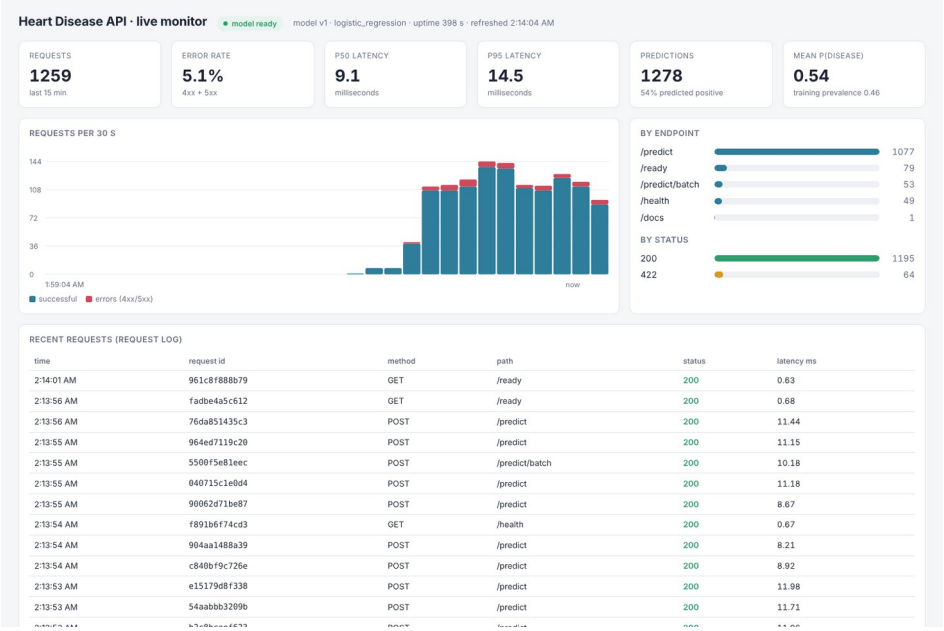


Figure 16. Built-in API monitoring dashboard during a load test (the error rate is the intentional 422 traffic).

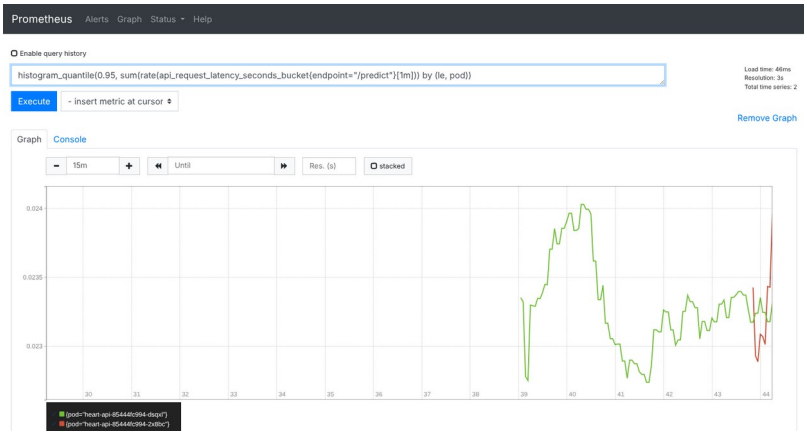


Figure 17. Prometheus: p95 latency of /predict per pod (about 23 ms) while the pods are scraped via pod discovery.

11. Repository, access and environment notes

Code repository: add the GitHub URL here after pushing (`git remote add origin <url> && git push -u origin main`). The first push triggers the GitHub Actions workflow; save screenshots of the run into `screenshots/`.

API access (local): run `docker run -p 8000:8000 heart-disease-api:1.0.0` and open `http://localhost:8000/docs`. On Docker Desktop Kubernetes, `kubectl apply -k k8s/` exposes the API at `http://localhost/` through the LoadBalancer. On Minikube, run `minikube tunnel` and use the Service's EXTERNAL-IP. Test with `scripts/smoke_test.sh <url>`.

How the evidence was produced. The screenshots were captured in a Linux build sandbox whose network allowed only package registries and GitHub source code:

- Docker Hub was blocked, so the image was built with `BASE_IMAGE` set to a local Ubuntu 24.04 + Python 3.12 image. The Dockerfile is otherwise unchanged.
- The cluster is single-node Kubernetes v1.33.1 compiled from the official source, with containerd, CNI bridge networking and kube-proxy.
- A minimal controller assigned the node IP to the LoadBalancer Service, the role Docker Desktop or `minikube tunnel` plays.
- No Ingress controller or Grafana image could be pulled. The Ingress object is therefore defined but unused, and dashboards were shown with Prometheus and the API's built-in page. Both run normally with `docker compose` or on Minikube.

Limitations and next steps:

- The dataset is small (303 patients from one hospital), so performance estimates have wide uncertainty and need external validation before clinical use.
- The 0.5 decision threshold could be tuned for higher recall.
- Drift monitoring currently tracks the prediction mix. Input-distribution checks (e.g. Evidently) and automatic retraining from the registry would be the next additions.